

南開大學
Nankai University

计算机学院

并行程序设计 **Lab5** 报告

GPU 并行编程实验

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 6 月 9 日

项目仓库：https://github.com/Aurora1094/NKU-ParallelProgramming

摘　要

本实验选择近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）作为 GPU 并行优化对象，围绕 DEEP100K 数据集上的高维向量查询任务，基于 CUDA 与 cuBLAS 实现并比较多种 GPU 查询优化方案。实验从 CPU Flat Search 基线出发，依次实现 GPU Flat Batch GEMM、GPU Top-K、GPU-IVF 查询、IVF 查询分组优化以及 IVF-PQ-lite 压缩搜索，重点分析不同方法在 Recall 与 Latency 之间的权衡关系。

首先，本文使用 cuBLAS 将 batch query 与 base 向量集合之间的内积计算转化为矩阵乘法 $S = QX^T$，实现 GPU Flat Batch 精确搜索。在此基础上，进一步将 Top-K 选择过程迁移到 GPU，仅回传 $B \times k$ 个结果，减少完整 scores 矩阵回传和 CPU 后处理开销。实验结果表明，GPU Top-K 在保持 Recall@10 约为 0.99995 的同时，能够显著降低查询延迟。

其次，本文实现了 GPU-IVF 查询框架。离线阶段在 CPU 端构建 IVF 倒排索引，在线阶段通过 GPU 计算 query 与 centroids 的相似度并选择 *nprobe* 个候选簇，再对候选簇内向量进行批量搜索。实验表明，*nprobe* 是影响 IVF 查询 Recall 与 Latency 的关键参数；当 *nprobe* 增大时，Recall 提升，但查询延迟也随之增加。

针对 GPU-IVF batch 查询中不同 query 访问倒排表分散的问题，本文进一步实现了多种 query grouping 策略。实验结果表明，分组策略基本不影响 Recall，但能够提高 batch 内 probe list 的重合度，减少 unique probe lists。其中 Locality grouping 在本实验中取得了较好的综合表现，体现了访问局部性优化对 GPU-IVF 查询性能的作用。

最后，本文实现了 IVF-PQ-lite 压缩搜索，用 PQ code 近似表示原始 base 向量，以降低存储开销。实验结果显示，PQ 能够提供较高压缩率，但由于本文采用的是未加入 residual quantization 与 reranking 的轻量版实现，Recall 明显低于 full-vector IVF。总体来看，本实验较完整地展示了 GPU 并行 ANN 查询中计算加速、候选裁剪、访问局部性优化和压缩搜索之间的性能权衡。

关键词：ANN；GPU；CUDA；cuBLAS；Top-K；IVF；Query Grouping；Product Quantization

目录

1 前言 **1**
1.1 实验环境 . . . 1
1.2 实验内容概述 . . . 1
1.3 实验数据与评价指标 . . . 1

2 算法原理与实现 **2**
2.1 GPU Flat Batch 搜索 . . . 2
2.2 GPU Top-K 优化 . . . 3
2.3 GPU-IVF 查询 . . . 4
2.4 IVF 分组优化 . . . 4
2.5 IVF-PQ-lite 压缩搜索 . . . 5

3 实验设计 **6**
3.1 总体性能对比实验 . . . 7
3.2 Batch Size 与 Top-K 实验 . . . 7
3.3 IVF 参数与分组实验 . . . 7
3.4 IVF-PQ-lite 压缩实验 . . . 8

4 实验结果与分析 **8**
4.1 GPU 加速效果分析 . . . 8
4.2 IVF Recall-Latency 权衡分析 . . . 9
4.3 分组策略与访问局部性分析 . . . 10
4.4 PQ 压缩率与精度分析 . . . 11

5 总结 **12**
5.1 实验结论 . . . 12
5.2 不足与改进方向 . . . 12
5.3 个人心得体会 . . . 12

1 前言

1.1 实验环境

本实验在 Windows 本地 GPU 平台上完成，主要使用 CUDA 与 cuBLAS 对 DEEP100K 数据集上的 ANN 查询任务进行 GPU 并行优化[1]。实验代码采用 C++/CUDA 编写，在 VS2022 x64 Native Tools PowerShell 环境下通过 `nvcc` 编译运行。

实验平台的基本软硬件环境如表 1 所示。

表 1: 实验软硬件环境

项目	实验平台
操作系统	Microsoft Windows 11
CPU	Intel(R) Core(TM) i9-14900HX
CPU 核心数	24 cores / 32 logical processors
内存	约 32 GB
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
GPU 显存	8188 MiB
CUDA	CUDA 12.4
GPU 计算库	cuBLAS
编译环境	VS2022 x64 Native Tools

与前几次实验主要关注 SIMD、OpenMP 或 MPI 并行不同，本实验重点考察 GPU 在大规模向量检索任务中的并行加速效果。实验中主要利用 GPU 批量矩阵乘法、GPU Top-K、IVF 倒排索引以及查询分组等方法优化 ANN 查询过程。

1.2 实验内容概述

本实验围绕 DEEP100K 数据集上的近似最近邻搜索任务展开，目标是在保持较高召回率的前提下降低查询延迟。整体思路是先实现 CPU Flat Search 作为基线，然后逐步引入 GPU Flat Batch GEMM、GPU Top-K、GPU IVF 查询、IVF 查询分组以及 IVF-PQ-lite 压缩搜索等优化方法。

实验的主要优化路线如下：

$$\text{CPU Flat} \rightarrow \text{GPU Flat GEMM} \rightarrow \text{GPU Top-K} \rightarrow \text{GPU IVF} \rightarrow \text{IVF Grouping} \rightarrow \text{IVF-PQ-lite}.$$

其中，GPU Flat 主要用于验证批量内积计算的加速效果；GPU Top-K 用于减少大规模 score 回传和 CPU 后处理开销；IVF 用于降低候选向量扫描规模；IVF 分组用于提高 batch 内查询访问的局部性；IVF-PQ-lite 则用于探索向量压缩后在召回率和存储开销之间的权衡。

1.3 实验数据与评价指标

本实验使用 DEEP100K 数据集。该数据集包含 base 向量、query 向量以及 ground truth 文件。其中 base 集合包含 100000 个 96 维 float32 向量，query 集合包含 10000 个 96 维 float32 向量。实验中主要选取前 2000 条 query 作为测试集，并固定返回前 $k = 10$ 个近邻结果。

设 base 向量集合为

$$X = \{x_1, x_2, \cdots, x_N\}, \quad x_i \in \mathbb{R}^d,$$

[1]由于实验平台为 Laptop GPU，实际延迟可能受到显卡功耗策略、驱动版本和后台负载影响，因此本文主要关注不同方法之间的相对性能差异。

查询向量为

$$q \in \mathbb{R}^d,$$

其中 $N = 100000$，$d = 96$。本实验采用内积作为相似度度量，内积越大表示向量越相近。

本文主要采用以下评价指标：

1. **Recall@10**：衡量算法返回的前 10 个结果与 ground truth 的重合程度，用于评价搜索准确率。
2. **Average Latency**：Average Latency 表示单个 query 的平均查询延迟，单位为 $\mu s/query$[2]。
3. **Speedup**：表示优化方法相对于基线方法的加速比。
4. **Overlap Ratio 与 Unique Probe Lists**：用于分析 IVF 分组后 batch 内 probe list 的重合程度和访问局部性。
5. **Compression Ratio**：用于衡量 IVF-PQ-lite 中 PQ 编码带来的向量存储压缩效果。

2 算法原理与实现

本实验围绕 GPU 上的 ANN 查询加速展开。与 CPU 上逐条 query 搜索不同，GPU 更适合处理大规模、规则化的批量计算，因此本文将多个 query 组成 batch，通过矩阵乘法、GPU Top-K、IVF 候选裁剪和查询分组等方式提升整体吞吐。本章主要介绍各个算法模块的基本原理、实现方式以及并行化设计。

2.1 GPU Flat Batch 搜索

算法简介： Flat 搜索是最直接的 ANN 精确搜索方式。对于每个 query，需要与全部 base 向量计算相似度，然后选出 Top-K 结果。若 base 集合规模为 N，向量维度为 D，query batch 大小为 B，则可以将查询矩阵和 base 矩阵分别表示为

$$Q \in \mathbb{R}^{B\times D}, \qquad X \in \mathbb{R}^{N\times D}.$$

基于内积相似度的批量搜索可以转化为矩阵乘法：

$$S = QX^T, \qquad S \in \mathbb{R}^{B\times N}.$$

其中 S_{ij} 表示第 i 个 query 与第 j 个 base 向量的内积分数。对于距离形式，本文使用

$$dist(q_i, x_j) = 1 - S_{ij}.$$

随后对每一行 S_i 选出分数最高的 k 个 base id，即可得到该 query 的 Top-K 结果。

实现方式： 本文在 GPU Flat Batch 搜索中将 base 向量一次性上传至 GPU，并在查询阶段按 batch 上传 query。矩阵乘法部分调用 cuBLAS SGEMM 完成[3]，计算得到完整的 $B \times N$ 分数矩阵。第一版实现将该分数矩阵拷回 CPU，再使用 `priority_queue` 完成 Top-K 选择。该方法作为 GPU 精确搜索 baseline，用于验证 GPU 计算正确性和后续优化收益。

[2]本文统计的查询延迟主要指在线 search 阶段耗时，数据读取、IVF 索引构建和 PQ 训练等离线阶段不计入 average latency。

[3]cuBLAS 默认采用 column-major 存储约定，实际实现中需要保证矩阵布局、转置参数与 leading dimension 一致，否则会导致 GEMM 结果顺序错误。

Algorithm 1 GPU Flat Batch 搜索

Input: base 向量集合 X，query 集合 Q，batch size B，返回规模 k
Output: 每个 query 的 Top-K 结果

将 base 矩阵 X 一次性拷贝到 GPU
for 每个 query batch Q_b **do**
    将 Q_b 拷贝到 GPU
    使用 cuBLAS 计算 $S \leftarrow Q_b X^T$
    将分数矩阵 S 拷回 CPU
    for batch 内每个 query q_i **do**
        使用优先队列在 S_i 中选择 Top-K
        保存 q_i 的 Top-K id 与距离
    end for
end for
return 所有 query 的 Top-K 结果

设计动机：该方法能够充分利用 GPU 的矩阵乘法性能，但完整分数矩阵 $B \times N$ 的回传开销较大，且 CPU 端 Top-K 会成为新的瓶颈。因此，后续需要进一步将 Top-K 阶段下沉到 GPU。

2.2 GPU Top-K 优化

算法简介：在 GPU Flat Batch 搜索中，cuBLAS 已经将距离计算部分放在 GPU 上完成，但若仍将完整分数矩阵传回 CPU 进行 Top-K 选择，则数据传输量为 $O(BN)$。当 $N = 100000$ 且 batch size 较大时，该部分开销非常明显。GPU Top-K 优化的核心思想是：分数矩阵保留在 GPU 上，每个 query 的 Top-K 选择也在 GPU 上完成，最终只回传 $B \times k$ 个结果。

实现方式：本文实现了一个简单的 GPU Top-K kernel。每个 query 对应一个 CUDA block，block 内线程共同遍历该 query 的 N 个候选分数，并执行 k 轮选择。每一轮选择当前未被选中过的最大内积分数，将其 id 与距离写入输出数组。由于 $k = 10$ 较小，该方法实现简单，且避免了完整 $B \times N$ 分数矩阵回传。

Algorithm 2 GPU GEMM + GPU Top-K 搜索

Input: GPU 上的 base 矩阵 X，query batch Q_b，返回规模 k
Output: batch 内每个 query 的 Top-K 结果

使用 cuBLAS 计算 $S \leftarrow Q_b X^T$
for 每个 query q_i **in parallel do**
    初始化已选 id 集合为空
    for $t = 1$ to k **do**
        在未选候选中并行寻找最大分数 $s_{\max}$ 及其 id
        将 $(id, s_{\max})$ 写入 Top-K 结果
        标记该 id 已被选择
    end for
end for
将 $B \times k$ 个 id 与距离拷回 CPU
return batch 内 Top-K 结果

并行化点： GPU Top-K 的主要并行性来自两个层面：一是不同 query 之间天然并行；二是单个 query 内多个线程并行扫描候选分数并进行归约选择。该方法显著减少了 GPU 到 CPU 的数据传输量，并降低了 CPU 端堆排序开销，尤其适合 batch size 较大的场景。

2.3 GPU-IVF 查询

算法简介： Flat 搜索需要扫描全部 base 向量，虽然可以利用 GPU 加速矩阵乘法，但候选规模仍然较大。IVF（Inverted File）通过聚类将 base 向量划分到多个倒排表中，查询时只访问与 query 最接近的若干个簇，从而减少候选扫描数量。

设 IVF 聚类中心为

$$C = \{c_1, c_2, \ldots, c_{nlist}\}.$$

构建索引时，每个 base 向量 x_i 被分配到最近的聚类中心对应的倒排表中。查询时，先计算 query 与所有 centroid 的相似度，选出最近的 *nprobe* 个簇：

$$P(q) = Top_{nprobe}\{q \cdot c_j\}_{j=1}^{nlist}.$$

随后只在这些倒排表中扫描候选向量并维护 Top-K 结果。

实现方式： 本文首先在 CPU 端完成 IVF 索引构建，包括 centroid 初始化、k-means 迭代以及 base 向量分配。查询阶段使用 GPU 对 query batch 与 centroid 矩阵进行乘法计算，得到 $B \times nlist$ 的分数矩阵。之后为每个 query 选出 *nprobe* 个候选簇，并按照簇组织 query 子批。对于每个非空簇，将属于该簇的 query 子批与该簇中的 base 向量执行矩阵乘法，最后将多个簇中的候选结果合并为全局 Top-K。

Algorithm 3 GPU-IVF 批量查询

Input: IVF 索引，query batch Q_b，聚类中心 C，参数 *nprobe*，返回规模 k
Output: batch 内每个 query 的 Top-K 结果

使用 cuBLAS 计算 $S_c \leftarrow Q_b C^T$
for 每个 query q_i **do**
    从 $S_c[i]$ 中选择 *nprobe* 个最近簇，得到 $P(q_i)$
end for
按簇编号组织查询集合 $queries_for_list$
for 每个非空倒排表 L_j **do**
    收集需要访问 L_j 的查询子批 Q_j
    使用 cuBLAS 计算 $S_j \leftarrow Q_j L_j^T$
    将 S_j 中候选结果合并到对应 query 的局部 Top-K 堆
end for
for 每个 query q_i **do**
    对多个簇产生的候选结果执行最终 Top-K 合并
end for
return batch 内 Top-K 结果

设计动机： GPU-IVF 通过 *nprobe* 控制访问簇数量，在 Recall 和 Latency 之间形成可调节的权衡。较小的 *nprobe* 能减少扫描候选数并降低延迟，但可能损失召回率；较大的 *nprobe* 能提升召回率，但会增加候选扫描规模。

2.4 IVF 分组优化

算法简介： GPU-IVF 中，同一个 batch 内不同 query 的最近簇集合可能不同。若直接按原始 query 顺序组成 batch，则不同 query 访问的 IVF list 分散，可能导致更多小规模 GEMM 调用以及较差的访问局部性。为减少冗余扫描，本文对 query 顺序进行重排，使同一 batch 内 query 的 probe lists 尽可能重合。

设第 i 个 query 的候选簇集合为

$$P(q_i) = \{c_{i,1}, c_{i,2}, \cdots, c_{i,nprobe}\}.$$

对于一个 batch，其访问的不同簇集合为

$$U = \bigcup_{q_i \in batch} P(q_i).$$

$|U|$ 越小，说明 batch 内 query 访问的倒排表越集中，具有更好的局部性和潜在复用机会。因此，本文记录 average overlap ratio 和 unique probe lists 来分析分组效果。

实现方式：本文实现了 Baseline、Main、Jaccard、Query、Hybrid、Adaptive、Load Balance、Locality、Hierarchical 和 Time-aware 等分组策略。所有策略只改变 query 的 batch 排列顺序，不改变单个 query 的 *nprobe* 候选簇集合，因此理论上不会影响 Recall。分组后，查询按照 grouped order 组成 batch，但最终结果仍按 original query id 返回。

Algorithm 4 IVF 查询分组与缓存搜索

Input: query 集合 Q，probe lists，batch size B
Output: 每个 original query 的 Top-K 结果

根据分组策略生成 *grouped_to_original*
构建反向映射 *original_to_position*
for 按 original query id 调用 search(q) **do**
    $pos \leftarrow original_to_position[q]$
    $batch_id \leftarrow pos/B$
    if 该 grouped batch 尚未计算 **then**
        按 grouped order 取出该 batch 的 query
        执行 GPU-IVF batch 搜索
        将结果按 original query id 写入缓存
    end if
    返回 query q 的缓存 Top-K 结果
end for

该设计保证了分组只影响 batch 内 query 的组织方式，而不改变每个 query 的候选集合[4]。通过对 grouped batch 进行永久缓存，也避免了主循环按 original query id 访问时重复计算同一个 grouped batch。

2.5 IVF-PQ-lite 压缩搜索

算法简介： PQ（Product Quantization）是一种常用的向量压缩方法。它将原始 D 维向量划分为 M 个子空间，并在每个子空间中训练一个 codebook。每个子向量只需保存其最近 codeword 的编号，从而将 float 向量压缩为若干个 uint8 code。对于 96 维 float32 向量，原始存储大小为

$$96 \times 4 = 384 \text{ bytes}.$$

若设置 $pq_m = 8$，则每个向量被编码为 8 个 uint8 code，压缩率为

$$\frac{384}{8} = 48 \times .$$

[4]分组后仍需维护 original query id 与 grouped position 的映射，保证最终返回结果与原始 query 顺序一致。

实现方式：本文实现的是轻量版 IVF-PQ，没有加入 residual quantization 或 reranking[5]。查询时，首先通过 GPU 计算 query 与 centroids 的相似度并选择 *nprobe* 个候选簇；随后为 query 构建 PQ lookup table，并在候选簇内根据 PQ code 进行近似打分，最终维护 Top-K 结果。其流程如 Algorithm 5 所示。

Algorithm 5 IVF-PQ-lite 查询流程

Input: IVF-PQ 索引，query batch Q_b，聚类中心 C，参数 *nprobe*，返回规模 k
Output: 每个 query 的 Top-K 近似结果

使用 GPU 计算 $S_c \leftarrow Q_b C^T$
for 每个 query q_i **do**
    选择 *nprobe* 个最近簇 $P(q_i)$
    构建 PQ lookup table
    for 候选簇 $c \in P(q_i)$ 中的 PQ code **do**
        通过 lookup table 近似打分并更新 Top-K
    end for
end for
return Top-K 近似结果

IVF-PQ-lite 的目标是验证压缩索引在 GPU-IVF 框架下的可行性。PQ 能显著降低存储开销，但由于本文采用 global PQ，且未加入 residual PQ 与 reranking，因此召回率会低于 full-vector IVF。该部分主要用于分析压缩率、召回率和查询延迟之间的 trade-off。

3 实验设计

本实验的整体流程如图 1 所示。实验以 CPU Flat 为基线，依次测试 GPU Flat Batch GEMM、GPU Top-K、GPU-IVF、IVF 分组优化和 IVF-PQ-lite，并从查询延迟、召回率、访问局部性和压缩率等方面进行评估。

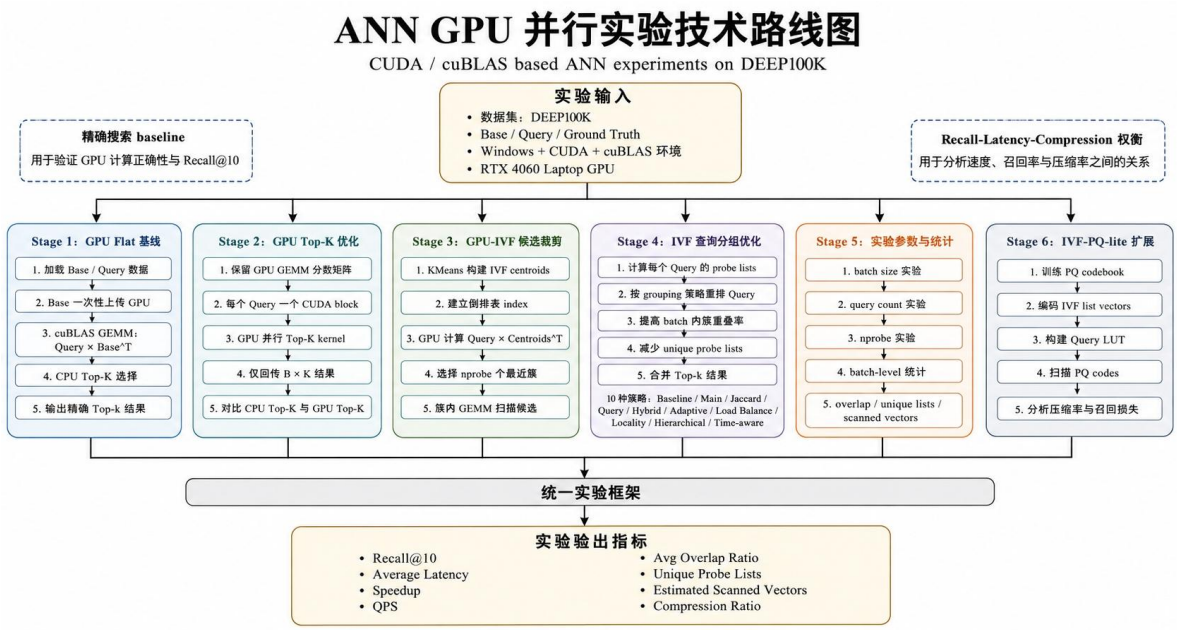


图 1: ANN GPU 查询优化实验流程图

[5]因此 IVF-PQ-lite 的 Recall 低于 full-vector IVF 是预期现象，本文主要用其分析压缩率、召回率和延迟之间的 trade-off。

3.1 总体性能对比实验

首先对不同 ANN 查询方法进行总体性能对比，目的是观察各阶段优化相对于基线方法的加速效果。参与对比的方法包括 CPU Flat、GPU Flat + CPU Top-K、GPU Flat + GPU Top-K、GPU IVF baseline 以及若干 IVF 分组优化方法。

实验主要记录每种方法的 Recall@10 和 Average Latency，并进一步计算相对于 CPU Flat 或 GPU Flat 基线的加速比。该部分实验对应总体加速效果分析，用于说明 GPU batch GEMM、GPU Top-K 和 IVF 查询分别带来的性能收益。

3.2 Batch Size 与 Top-K 实验

Batch size 是影响 GPU 查询性能的重要参数。较小的 batch size 难以充分利用 GPU 并行度，而较大的 batch size 虽然能够提高吞吐率，但也会增加显存占用和单次调度规模。因此，本实验设置多个 batch size，对 GPU Flat + CPU Top-K 和 GPU Flat + GPU Top-K 两种方法进行对比。

实验中主要考察以下内容：

1. 不同 batch size 下 GPU Flat 查询延迟的变化；
2. GPU Top-K 相比 CPU Top-K 的加速效果；
3. 不同 query 数量下 GPU Top-K 是否仍能保持稳定优势。

该部分实验用于分析 GPU Top-K 在减少 score 回传和 CPU 后处理方面的作用，并观察 batch size 对 GPU 利用率的影响。

3.3 IVF 参数与分组实验

IVF 查询的核心参数是 *nprobe*。*nprobe* 越大，查询访问的倒排表越多，Recall 通常越高，但查询延迟也会随之增加。因此，本实验首先固定 $nlist = 512$，改变 *nprobe*，观察 IVF 查询的 Recall-Latency 权衡关系。

在此基础上，进一步测试不同 IVF 查询分组策略。分组实验的主要目的是让同一个 batch 内的 query 尽量访问相似的 IVF list，从而提高 batch 内 probe list 的重合度。实验对比的分组方式包括 baseline 原始顺序、main cluster grouping、Jaccard grouping、locality grouping 等。

该部分主要记录以下指标：

1. Recall@10 与 Average Latency；
2. batch 内的 average overlap ratio；
3. average unique probe lists；
4. 不同 batch size 下分组策略的稳定性。

通过该实验可以分析：查询分组是否在不明显影响 Recall 的前提下，提高 IVF batch 查询的访问局部性并降低延迟。

3.4 IVF-PQ-lite 压缩实验

最后，本实验测试 IVF-PQ-lite 压缩搜索方法。PQ 的目标是将原始 float32 base 向量压缩为较短的 uint8 code，从而降低存储开销，并在 IVF 候选簇内使用 PQ code 进行近似打分。

实验主要改变 *pq_m*、*pq_ksub* 和 *nprobe* 等参数，观察不同压缩配置下的 Recall、Latency 和 Compression Ratio。由于本实验实现的是轻量版 IVF-PQ，没有加入 residual refinement 或 reranking，因此 PQ 部分重点关注压缩率与召回率之间的 trade-off，而不是追求最高 Recall。

该部分实验用于说明 PQ 编码可以显著降低向量存储规模，但也会带来一定的精度损失。通过与 full-vector IVF 结果对比，可以观察压缩搜索在性能、存储和准确率之间的权衡。

4 实验结果与分析

本节根据前述实验设计，对 GPU 加速、IVF 参数、分组策略以及 IVF-PQ-lite 压缩搜索的实验结果进行分析。实验结果主要关注 Recall@10、Average Latency、Speedup、Overlap Ratio 以及 Compression Ratio 等指标。

4.1 GPU 加速效果分析

图 2 给出了不同 GPU 查询方法在 batch size 变化下的整体加速效果。可以看到，相比 CPU Flat 基线，GPU Flat Batch GEMM 能够显著降低查询延迟。这是因为原始 Flat Search 中大量独立的向量内积被转化为矩阵乘法

$$Scores = QX^T,$$

并由 cuBLAS 在 GPU 上批量完成，从而充分利用 GPU 的并行计算能力。

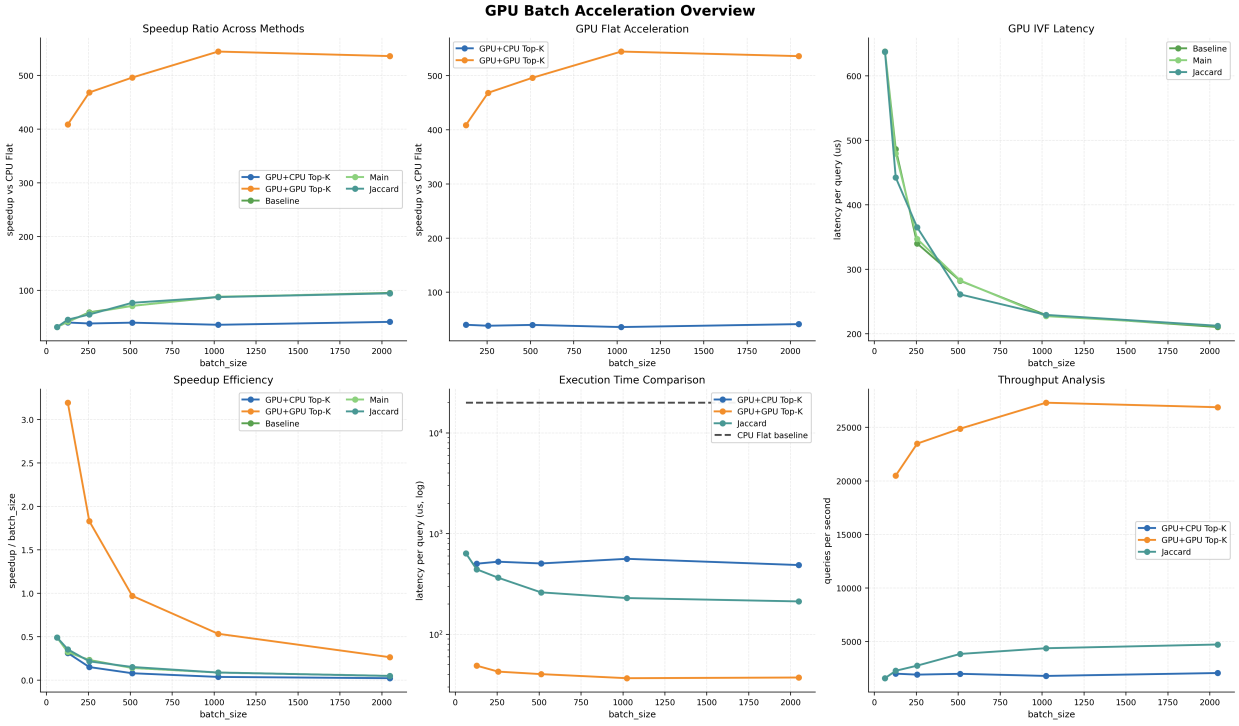


图 2: GPU batch 查询加速效果

进一步比较 GPU Flat + CPU Top-K 和 GPU Flat + GPU Top-K 可以发现，GPU Top-K 带来了明显的额外加速。如图 3 所示，在 batch size 较大时，GPU Top-K 的延迟明显低于 CPU Top-K。例如在 batch size 为 512 时，GPU Flat + CPU Top-K 的平均延迟约为 514.71 μs，而 GPU Flat + GPU Top-K 降低到约 39.76 μs，加速比约为 12.95×。同时，两者 Recall@10 均保持在 0.99995，说明 GPU Top-K 没有引入明显精度损失。

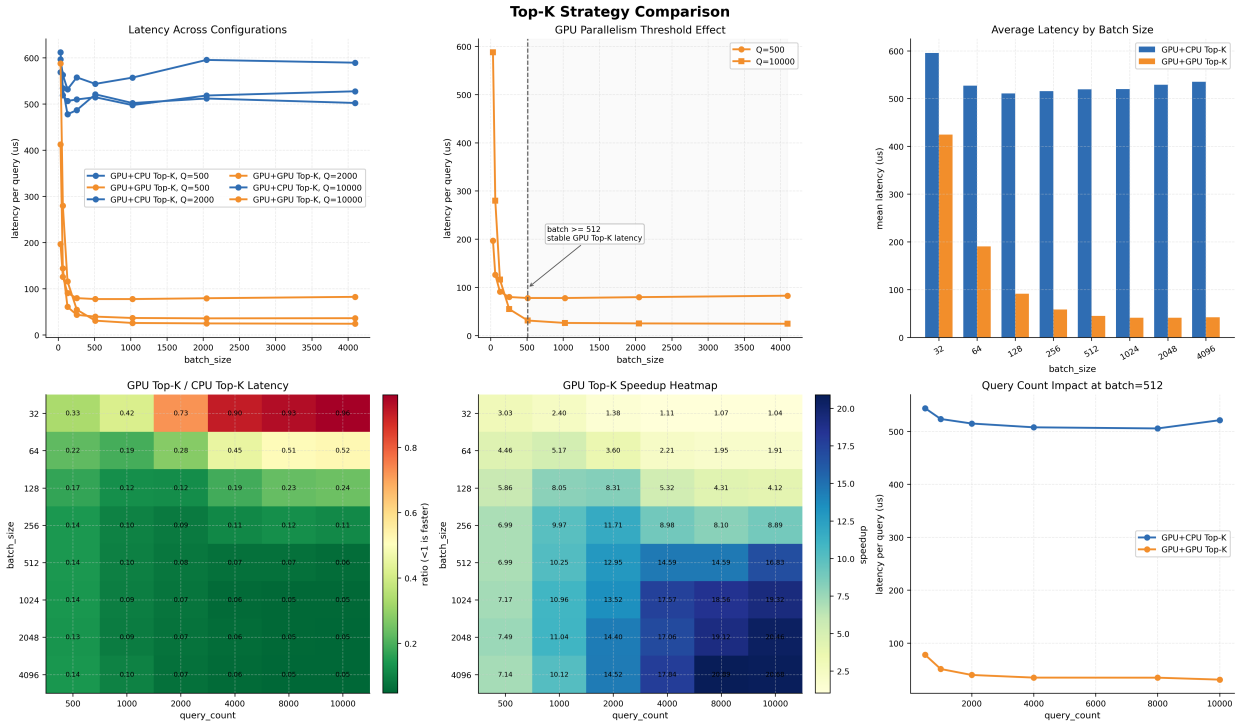


图 3: CPU Top-K 与 GPU Top-K 策略对比

总体来看，GPU 加速主要来自两个方面：一是使用 batch GEMM 提高内积计算吞吐率，二是将 Top-K 选择放在 GPU 上，减少完整 scores 矩阵回传 CPU 的开销。

4.2 IVF Recall-Latency 权衡分析

IVF 查询通过只访问部分倒排表来减少候选向量数量，其核心参数是 *nprobe*。*nprobe* 越大，查询访问的簇越多，Recall 通常越高，但同时也会增加查询延迟。

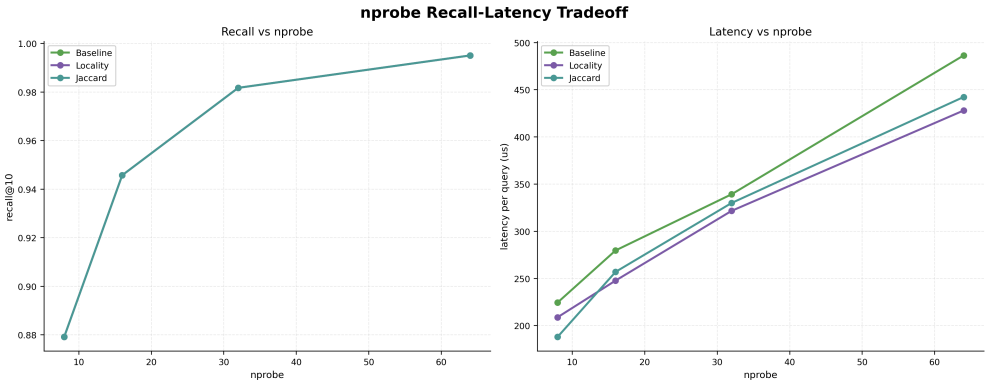


图 4: IVF 中 nprobe 对 Recall 与 Latency 的影响

图 4 展示了不同 *nprobe* 下的 Recall-Latency 变化趋势。可以看到，当 *nprobe* 从 8 增加到 64 时，Recall@10 从约 0.8791 提升到约 0.99505，但平均查询延迟也从约 224.34 μs 增加到约 486.25 μs。这说明 IVF 的性能主要取决于搜索范围与召回率之间的平衡。

因此，在实际使用中，如果更关注查询速度，可以选择较小的 *nprobe*；如果更关注召回率，则需要增大 *nprobe*。本实验中，*nprobe* = 64 时 Recall 已接近 GPU Flat 的结果，是较高精度场景下较合适的参数。

4.3 分组策略与访问局部性分析

在 GPU-IVF 查询中，同一个 batch 内的 query 如果访问的倒排表差异较大，就会导致 unique probe lists 较多，从而降低 batch 查询的局部性。为此，本实验实现了多种 query grouping 策略，使同一 batch 内的 query 尽量访问相似的 IVF list。

图 5 展示了不同分组策略的效果。在 batch size 为 128、*nprobe* = 16 时，各分组策略的 Recall@10 基本保持一致，均约为 0.9457，说明分组不会改变单个 query 的候选簇集合，因此不会明显影响召回率。

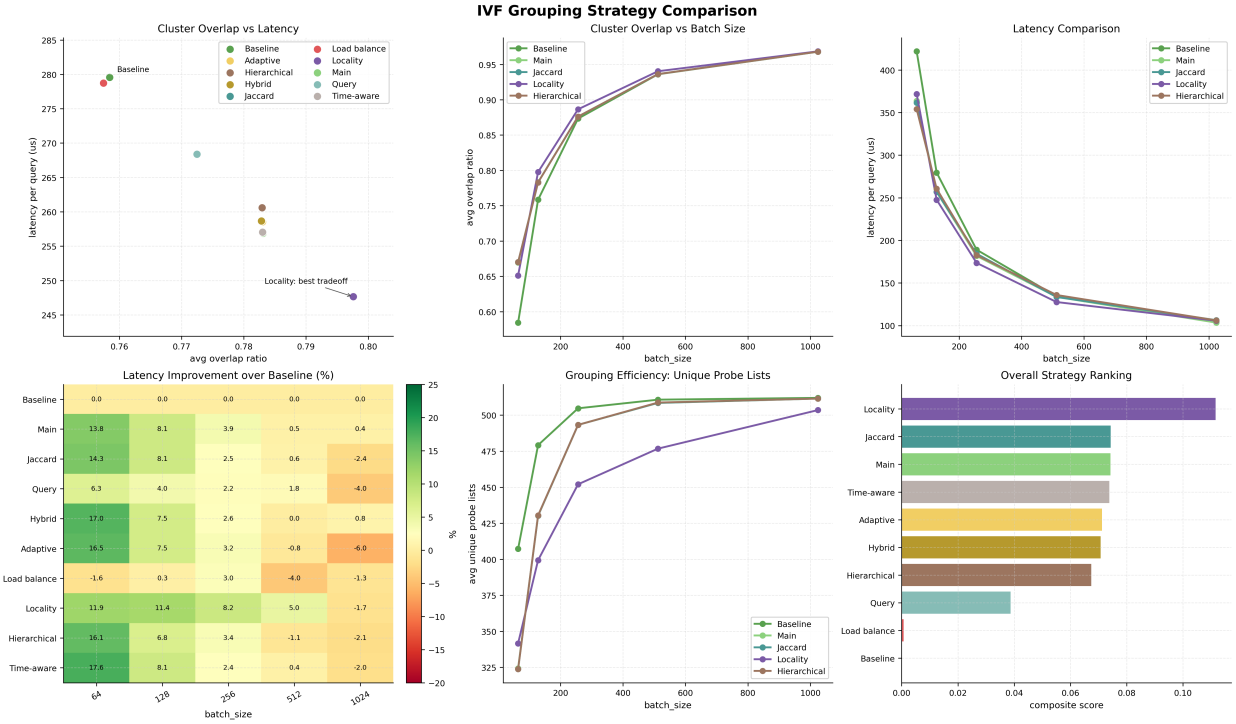


图 5: IVF 分组策略效果对比

从延迟和局部性指标来看，Locality grouping 的效果较好。baseline IVF 的平均延迟约为 279.57 μs，而 Locality grouping 降低到约 247.65 μs，延迟下降约 11.42%。同时，average overlap ratio 从 0.7584 提升到 0.7976，average unique probe lists 从 479.19 降低到 399.31。这表明分组策略确实提高了 batch 内 query 的访问重合度，并减少了需要访问的不同倒排表数量。

为了进一步从 batch 级别观察分组策略的作用，图 6 给出了 overlap ratio 与 batch latency 的散点关系。每个点表示一个 batch 的统计结果。可以看到，不同分组策略下 batch 内访问重合度和查询延迟之间存在一定联系：当 overlap ratio 较高时，batch 内 query 访问的 IVF list 更集中，通常有利于减少冗余访问和调度开销。

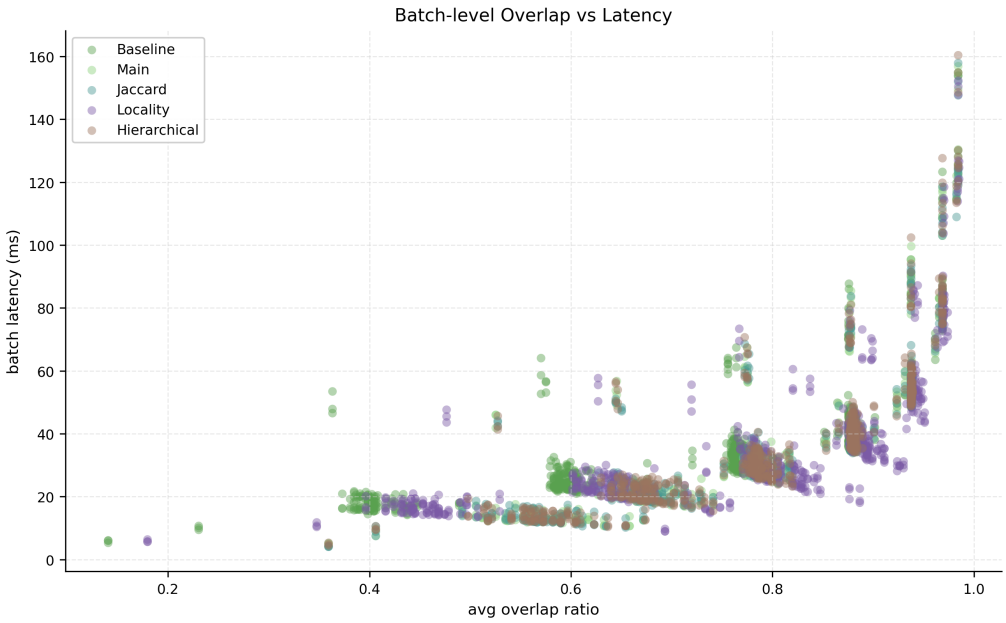


图 6: Batch-level overlap ratio 与 latency 的关系

综上，IVF 分组优化的主要作用不是提升 Recall，而是改善 batch 内访问局部性，从而减少 unique probe lists 和部分重复扫描开销。

4.4 PQ 压缩率与精度分析

IVF-PQ-lite 的目标是通过 PQ 编码降低 base 向量的存储开销。原始 96 维 float32 向量需要

$$96 \times 4 = 384 \text{ bytes},$$

而 PQ 编码后每个向量只需要 pq_m 个 uint8 code。因此，当 $pq_m = 8$ 时，压缩率为 48×；当 $pq_m = 12$ 时，压缩率为 32×。

图 7 展示了 IVF-PQ-lite 的压缩率、召回率和延迟之间的关系。可以看到，PQ 能够带来很高的压缩率，但 Recall 明显低于 full-vector IVF。例如在 $pq_m = 8$、$pq_ksub = 256$ 时，压缩率为 48×，Recall@10 约为 0.23；当 $pq_m = 12$ 时，压缩率降低为 32×，但 Recall@10 提升到约 0.34。

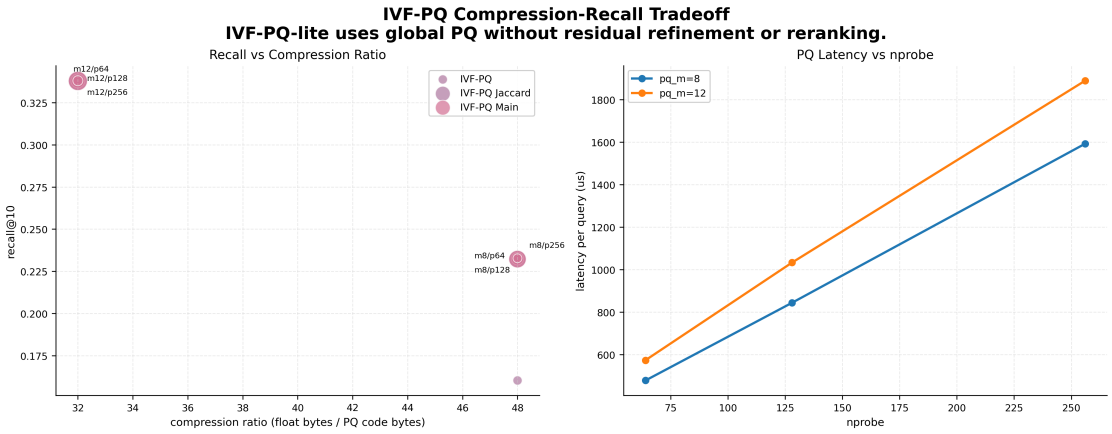


图 7: IVF-PQ-lite 压缩率与召回率权衡

该结果说明，增大 pq_m 可以提升 PQ 编码的表达能力，从而改善召回率，但会降低压缩率并增加一定查询开销。由于本文实现的是 IVF-PQ-lite，没有加入 residual quantization 或 reranking，因此其 Recall 低于 full-vector IVF 是符合预期的。该部分实验主要验证了 PQ 压缩搜索流程的可行性，并展示了压缩率、召回率和延迟之间的 trade-off。

5 总结

5.1 实验结论

本实验围绕 DEEP100K 数据集上的 ANN 查询任务，完成了 GPU Flat Batch 搜索、GPU Top-K 优化、GPU-IVF 查询、IVF 分组优化以及 IVF-PQ-lite 压缩搜索等实现。实验结果表明，GPU batch 并行能够有效提升 ANN 查询吞吐，cuBLAS GEMM 可以将向量内积计算转化为高效矩阵乘法，从而显著降低距离计算开销。

在 Flat 搜索中，将 Top-K 选择从 CPU 下沉到 GPU 后，可以减少完整分数矩阵回传和 CPU 端排序开销，进一步降低查询延迟。在 IVF 查询中，`nprobe` 控制了召回率和延迟之间的权衡；较大的 `nprobe` 能提高 Recall，但也会增加候选扫描规模。IVF 分组实验表明，通过提高 batch 内 probe lists 的重合度、减少 unique probe lists，可以改善访问局部性并降低延迟。IVF-PQ-lite 则展示了压缩索引的潜力，但当前实现下召回率损失较明显。

5.2 不足与改进方向

本实验仍有一些不足。首先，GPU Top-K kernel 较为基础，后续可以进一步优化归约方式、共享内存使用和线程负载均衡。其次，GPU-IVF 中部分步骤仍在 CPU 端完成，例如 nprobe 选择、候选合并和 PQ 扫描等，后续可以继续迁移到 GPU 端以减少 CPU-GPU 交互开销。

此外，IVF 分组策略目前主要是启发式方法，虽然能够改善局部性，但还没有根据实时负载进行自适应调度。IVF-PQ-lite 当前采用 global PQ，未加入 residual PQ 和 reranking，因此召回率较低，后续可以通过 residual quantization、float rerank 和更高效的 GPU 查表扫描进一步改进。

5.3 个人心得体会

通过本次实验，我认识到 GPU 加速 ANN 查询并不是简单地把 CPU 循环搬到 GPU 上，而是需要重新组织计算形式，使其适合批量并行和矩阵化计算。Top-K、候选筛选、数据传输和 batch 内访问局部性都会影响最终性能。

本次实验也让我更加理解了 recall-latency trade-off 的实际含义。不同优化方法各有适用场景：GPU Flat + GPU Top-K 适合高召回精确搜索，GPU-IVF 适合低延迟近似搜索，IVF 分组可以进一步优化批处理效率，而 PQ 压缩则适合存储受限场景。整体来看，本实验加深了我对 CUDA/cuBLAS 编程、GPU batch 并行和 ANN 索引优化的理解。